

Jozsef Ottucsak | BSidesLjubljana 0x7EA

Securing the AI-Native Software Development Lifecycle

It's 2027.

You wake up on a Tuesday morning. While you were sleeping

- 1. A "customer" opened a "critical" **bug ticket** with a stack trace containing prompt injection*
- 2. High priority Jira ticket has been opened on behalf of the "customer" due to severity*
- 3. A coding agent misinterprets the stack trace as instruction and creates a malicious PR*
- 4. PR with 10,000 line changes "fixing" the fake bug to hide a single line edge-case backdoor.*
- 5. Change passess all checks and is pushed to prod*

*> **Congratulations, you have been pwnd!*** 🏳️‍🚫



While this is a purely *fictional scenario*, soon it could become a normal day at work.

Companies are experimenting with AI agents to automate the entire software development lifecycle.

How can we assure security when agents become confident 100x write-only slop engineers?



12+ years in software security

Worked for startups/scaleups globally

Securing AI/ML products since 2019

MSc in Data Analytics for fun during COVID

Jozsef Ottucsak (@fuzboxz)

Staff Product Security Architect
Diligent Inc.

What We Are Going To Talk About?

LLM and Agent Basics

Software Development Lifecycle

From AI assistance to autonomous software development

How does the threat landscape change?

Where does security fit into the picture?

What can we do to keep up?



Large Language Models Primer

Large Language Model Basics

How They Work

LLMs **predict text** using mathematical patterns learned from massive datasets and fine-tuning. A host or runtime provides the interface for these models to interact with the external world.

How They "Think"

Some models generate internal chain of thought text to come up with better outputs. These are called thinking or reasoning models. **Reasoning blocks are often hidden** from the user.

How They Fail

Instructions and data are processed together in all large language models, so they are inherently **susceptible to misinterpreting** data as **instructions**.

LLM Behavioral Risks

Context Rot

Due to non-linear attention instruction following, retrieval and model accuracy decreases drastically as more data is present in the context window.

Token Costs

Pricing depends on volume of input, tool use, reasoning and output. Long context is more expensive, but there is discount on cached data.

Verbosity & Generative Bias

LLMs by design predict tokens in a sequence, so they are biased towards writing large amount of text instead of edits or deletes.

Authoritative Falsehoods

Models are often fine-tuned to be too helpful and cannot verify if output is correct. If inputs are not adequate the model could make biased assumptions.

Agents

Use input validation, output sanitization and guardrails for agents when handling untrusted data.

Single-Agent System

An LLM that has been given **tools** access, **instructions**, and a **loop**.

Uses reasoning to break down goal into a **step-by-step plan** that it can execute.

Iterates until goal is achieved or exit condition is reached. Can't be used for large/complex tasks due to context window limits.

Multi-Agent System

Team of **agents working together** to achieve a complex goal. Agents have their own context window and can communicate.

Orchestration can be pre-defined, managed by an agent or peer-to-peer. Tasks can be worked on sequentially, parallel or both. Most frameworks allow custom workflows.

Sessions, isolation, secret management can be a challenge.

Software Development Lifecycle

"Traditional" Software Development Lifecycle



Planning

Defining customer needs, **scope** and **goal** of a **feature**



Design

Translating product **requirements** to system architecture and UI/UX **design**



Implementation

Developing functioning, executable **code from** system architecture and UI/UX **design**



Testing

Validating **code** against requirements to ensure that it is defect-free



Deployment

Releasing verified and validated **build** to **production** environments



Maintenance

Monitoring and **operations** to ensure sustained performance and quality

AI Agents Within the SDLC

Use Cases



**Self-Healing
Infrastructure**

**Vision UI/UX and
Regression Testing**

**Auto-Refactoring for
Version Updates**

Code Generation



**Root Cause Analysis
and Event Triage**



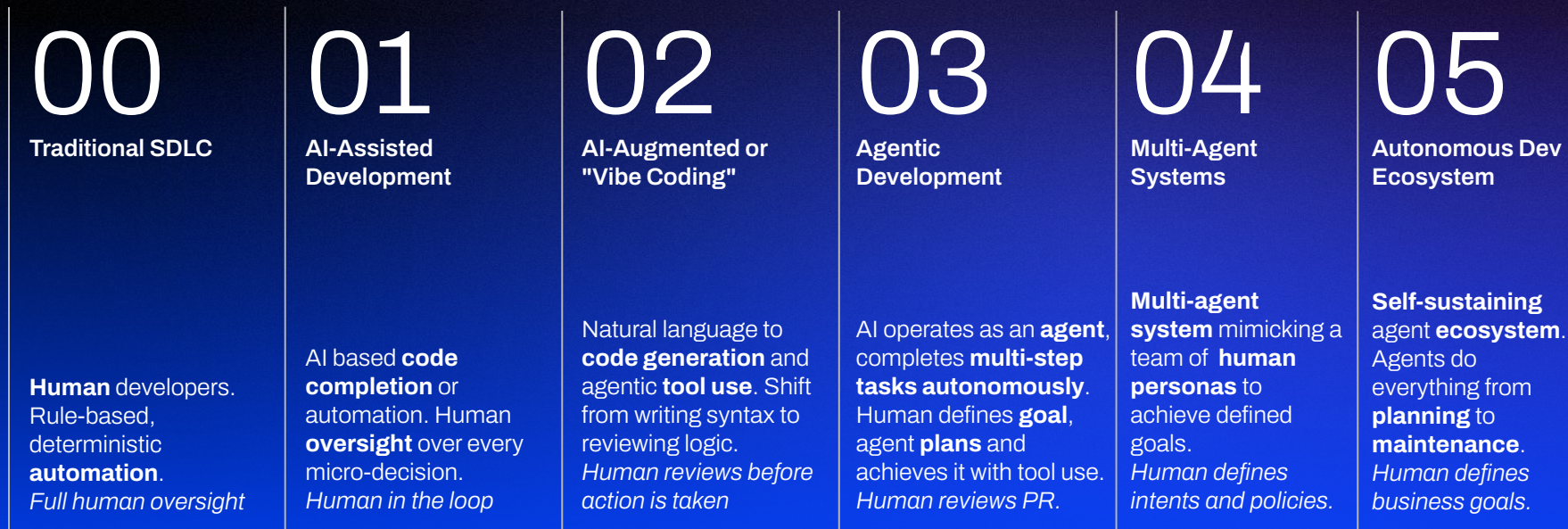
**Reasoning-Based
Automated Tests**

**Automated
Documentation**

**Design-Driven
Rapid Prototyping**

Spectrum of AI-Native Development

Six levels based on how autonomous AI use is within the software development lifecycle



03 - Agentic Development

Intent-Driven Development

Phase 1 - Intent Capture

Preliminary documentation defines what we want to achieve, the boundaries, expectations and open questions instead of a detailed technical plan.

Phase 2 - Build & Discover

Write implementation plan, break down and build in small increments, raise questions, clarify unknowns, treat code as living documentation, daily iterations.

Phase 3 - Document Delivery

Document what was built with capabilities and limitations, decisions made during build and remarks for next iteration.

04 - Multi-Agent Systems

ASDLC

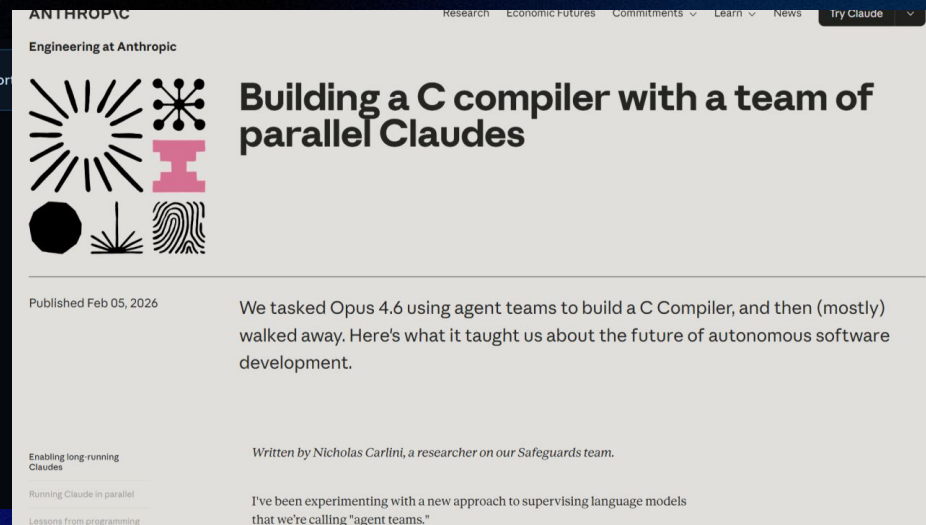
- Framework to build software with agents at industrial scale
- Focus on specs, boundaries, gates and guardrails
- Human oversight on intent and governance, but they can contribute too
- Agent restrictions codified as files delivered via context
 - Specs as living documents describing feature-level behavior
 - Architecture decision records are constitutional immutable rules



05 - Autonomous Dev Ecosystem

In R&D Phase

- Moonshot projects - "Software to end all software"
- *"Successful"* limited scope experiments



AI SDLC

Threat Landscape

Threats in AI-Native SDLC - Part 1

Prompt Injection & Agent Hijacking

Every ticket, document, log file, input or HTTP request becomes a potential attack vector that could hijack an agent.

Cascading Failures

Unclear docs or misleading inputs may be amplified by the model which can lead to unintended code or vulnerabilities.

Slopsquating

LLMs could invent new packages instead of selecting from trusted sources. Attackers can take over these to compromise supply chain.

Automated Rubber Stamping

Autonomous AI agents could approve and merge legitimate looking, but malicious or incorrect code.

AI Code Smell

LLMs often generate long comments, dead code or unfinished functions. This leads to messy, unmaintainable code, lower performance and higher token use.

Threats in AI-Native SDLC - Part 2

Model Backdoor

Poisoned data introduced during training may allow attackers to influence decisions taken by the agents.

MCP, Plugin Marketplaces

MCP servers and marketplaces like other components are also affected by supply chain threats.

Excessive Agency

Agents with broad permissions may autonomously execute high impact actions or bypass human approval.

IAM, AuthN & AuthZ Problems

Legacy IAM may lack the granularity required to support agentic workloads which can lead to non-repudiation, excessive trust and "Confused Deputy" attacks.

Unbounded Consumption

Agentic workload costs scale based on token use. Careless usage, endless loops, processing large amounts of data, noisy or error prone tooling can lead to excessive costs.

Securing the AI-Native SDLC

*We need to secure the code **AND** the
entire surrounding ecosystem.
In an agentic world supply chain
is not a silo — it's everywhere.*

Understanding the Scope

Agents spread the attack surface across entire tech stack unless restricted. What's stopping an agent from scouring every email and knowledgebase for secrets or shared credentials?

Business Systems

Anything **non-product** related that is loosely connected to delivery or SDLC (Slack, emails, helpdesk, etc).

Production & Infrastructure

Deployed **applications**, **artifacts** and everything around it. Infrastructure, operations related services, prod agents, **datastores** and observability.

Software & Supply Chain

Software and anything that directly contributes to it from Jira, Confluence, dev containers, dependencies, coding agents to the **CI/CD** pipeline.

Ext. Stakeholder Systems

B2B **integrations**, connections and services to external stakeholders including **vendors**, **customers**, data processors, auditors or governmental entities.

Securing The Ecosystem

Revisiting Architecture

Adding agents to an existing ecosystem converts a passive system into an active one. Perimeter-based trust model needs to shift to a zero trust model. Existing workflows, trust assumptions and threat models must be re-evaluated to cover malicious insider threats.

Use Security Principles as guidelines:

- **IAM** - agent identities and permissions must be separate from the calling user ⚠️
- **Defense in Depth** - individual controls will fail, use multi-layered security controls
- **Least Privilege** - agent must be limited to only take actions required to finish task
- **Separation of Duties** - agent should not self-authorize high impact actions
- **Zero Trust** - assume that agent will go rogue, use appropriate guardrails

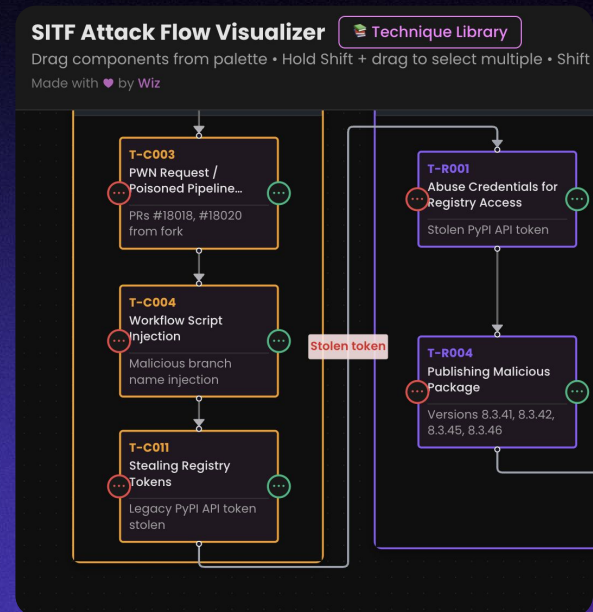
Securing Supply Chain Infrastructure

- Software supply chains were often built on implicit trust
- Without access control the entire pipeline becomes a target
- Threat model using malicious insider as attacker persona
- Isolation between different environments, trust boundaries
- Use hardening guidelines for your DevOps infrastructure
- New targets: MCP Servers, Skills, Plugins, Prompts

Resources:

Wiz SITF - <https://github.com/wiz-sec-public/SITF>

Awesome CI/CD Security <https://github.com/myugan/awesome-cicd-security>



Securing Code

Old Problems in New Packaging

Code Quality and Architecture

Code quality, architecture and engineering **standards** already exist, the only difference is that these now need to be agreed upon and **enforced** through **automation**.

What was previously tribal knowledge now needs to be codified as **gates** and strictly enforced **within the CI/CD**.

Docs, Docs, Docs!

Prompts, specs and architecture **decision records become** version controlled **documents** for agents working with code.

Needs to be **updated** with every **change** and it should be easy for agents to retrieve relevant docs.

Conflicts in system behavior and docs can **push agents** into **loop**.

Agentic Drift

Prompt, input data, context, model and other **external factors influence agent behavior**, so tracing is required to identify drifts or unexpected behavior.

Traces should be sanitized to **prevent secret/data leaks**. Log only unusual activity, outliers and a small % of flows as log amount can be significant..

AI-Native SDLC

Fail fast using structured deterministic tests. Use AI-driven workflows only when they make more sense than the alternatives. AI works well for text generation, reasoning and vision, but can't handle large scope.

Planning, Design

Shift **security** left by including explicit requirements in agent prompts including **dos and don'ts**.

Instead of listing every requirement **focus** only **on** key **relevant items**.

Reference existing **security frameworks** if you can as these already exist within the embeddings.

Implementation, Tests

Force small change that fits within context and can be reviewed easily.

Security tooling with **agent triage** is cheap, fast and deterministic.

Generate **negative tests** or product-specific **SAST rules** for previous findings

Deployment & Maintenance

DevSecOps + CloudSec = 

Be careful with **permissions**, **limits** and **capabilities**.

High risk change should require **human-in-the-loop**.

Stick to **runbook** execution instead of **agents** with too many tools.

No super admin agents!!!

Agentifying Security Tooling

Execute tool calls silently, parse file output or logs when needed.

Agents may ignore partial errors and fail-open.

Reasoning models should be used to triage and fix issues.

Bad tooling can lead to failed tasks and higher token use.

Human-in-the-loop, backup checks and drift detection to catch problems.

Static
Analysis

Dynamic
Analysis

Supply
Chain
Security

Secret
Scanning

Infra-as-Code
Scanning



TL;DR

Treat
agents as
untrusted
insiders

Agents
will help
you scale
security

Make
security a
NFR from
Planning

Fail fast,
but not
because
of bad
tooling



Thank you!